本章路線

這章要回答的核心問題是：作業系統如何表示、切換與協調彼此獨立又可能合作的執行單元？

前置知識。 CPU 暫存器、記憶體位址空間、中斷與 system call。

學習目標

- 追蹤 process state 與 PCB 內容
- 區分排程器、dispatcher 與 context switch 成本
- 設計具明確 blocking、buffering 與終止語意的 IPC

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

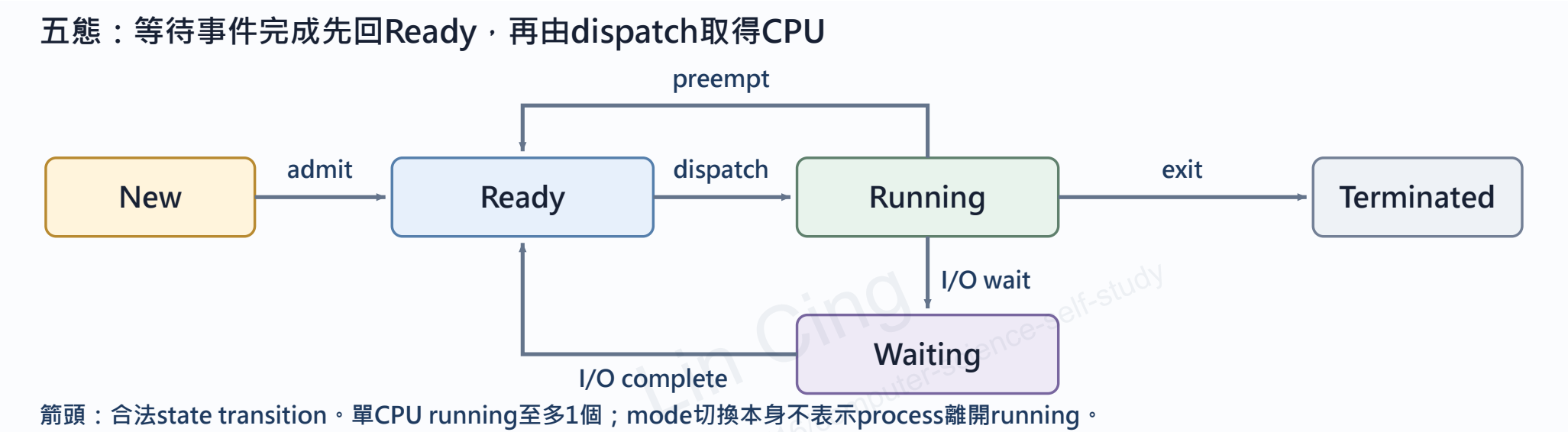
程式、行程與 PCB 的差別

程式（program）是可執行檔中的指令與資料；行程（process）是程式執行時的實例，具有身分、資源與執行狀態。同一程式啟動兩次，就有兩個不同的行程。

項目	P100	P101	可以共享嗎？
PID、PC、暫存器 r1	100、0x40、7	101、0x120、3	執行狀態各自獨立
位址 0x5000 的私有計數器	0 → 1	0 → 1	若未建立共享映射，兩次遞增不會相加
檔案描述符 fd 3	引用開啟檔案 A 的紀錄	引用開啟檔案 B 的紀錄	描述符數字相同，對象不一定相同
程式碼	同一程式的唯讀指令	同左	可以共用實體頁面，不表示共用執行狀態

核心用 PCB（process control block，行程控制區塊）追蹤行程身分、狀態、排程資訊，以及記憶體和 I/O 資源的引用。PCB 可指向頁表和開啟檔案的紀錄，但不必把整個 heap 存在 PCB 裡。

五態模型：每次轉移的原因



先假設一顆 CPU，而且每個行程只有一條執行流：同一時刻最多一個行程處於 running。**Ready** 表示已具備執行條件、正在等待 CPU；**waiting** 表示還在等待某個事件，不能直接參與排程。

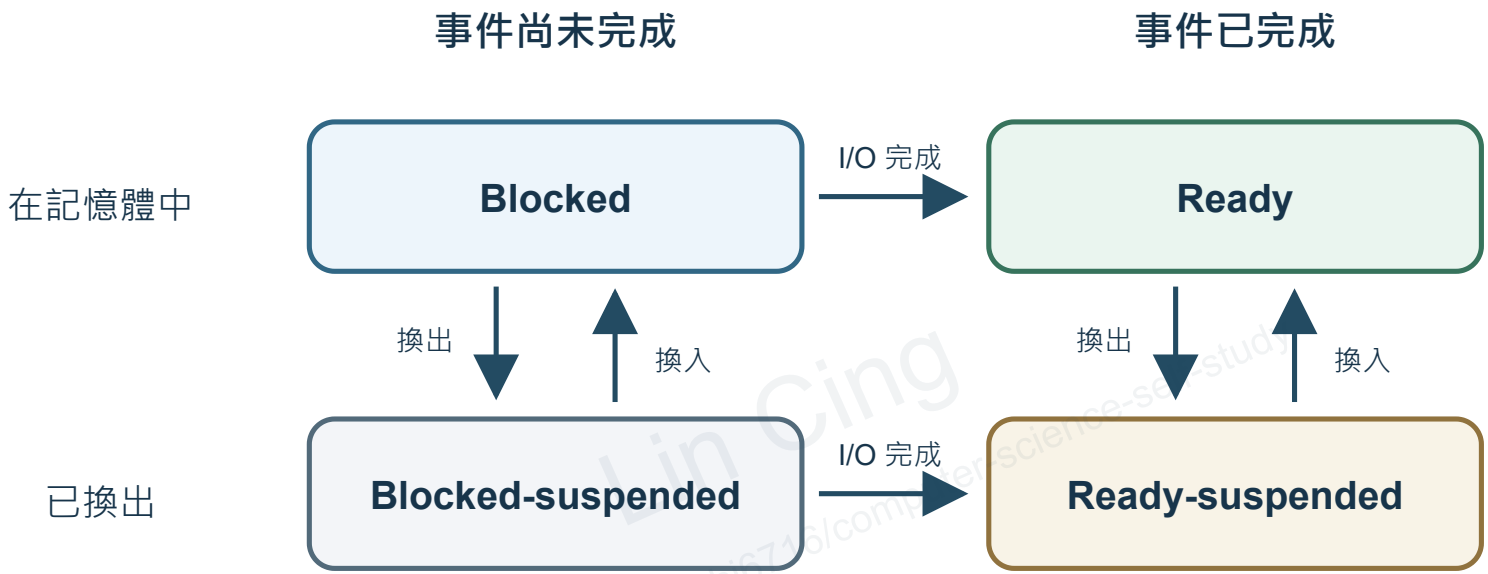
行程獲准進入系統後由 new → ready；派遣時由 ready → running；被搶占時由 running → ready；等待 I/O 時由 running → waiting；事件完成後由 waiting → ready；結束時由 running → terminated。

五態模型：每次轉移的原因 （續）

I/O 完成只使等待者回到 ready，何時真正執行仍由排程策略決定。在單 CPU 模型中，中斷處理常式可以喚醒等待者；等待者不需要先自己執行。此模型省略實際系統的細分狀態，terminated 也不代表所有紀錄立刻消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

在記憶體外等待：七態模型



Ready → dispatch → Running；Ready-suspended 必須先換入

這是教學用 whole-process swapping 模型：**blocked** 表示事件尚未完成；**suspended** 表示行程暫不具備在記憶體中執行的資格。兩者互相獨立。現代系統的虛擬記憶體可逐頁回收，不一定把整個 process 換出。

事件完成不等於立刻執行

時刻	事件	A 的狀態	能否分配 CPU？
0	A 發出 I/O	blocked	否，事件未完成
1	排程器換出 A	blocked-suspended	否，事件未完成且已換出
2	I/O 完成	ready-suspended	否，仍須載入
3	換入 A	ready	尚未執行，等待選取
4	dispatcher 選取 A	running	是

另一個 process 可在時間 1–3 執行。若先換入再等到 I/O 完成，則路徑是 blocked-suspended → blocked → ready；兩條路徑最後條件相同，順序不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

行程切換要保存什麼？

假設 P100 正在執行，PC＝0x40、r1＝7、SP＝0x9000；P101 先前保存的值是 PC＝0x120、r1＝3、SP＝0xA000。中斷或核心入口先依處理器架構保存必要狀態；核心再保存 P100 的執行上下文，載入 P101 的上下文及記憶體管理狀態。



行程切換要保存什麼？ （續）

時刻	CPU 正在執行	狀態
切換前	P100	PC＝0x40、r1＝7、SP＝0x9000
核心保存 P100	核心	將 P100 的可恢復狀態與它的紀錄關聯
核心載入 P101	核心 → P101	載入 P101 的 0x120、3、0xA000
稍後再選中 P100	P100	恢復 P100 自己的狀態

實際保存的 PC 及是否重試指令，取決於架構與事件種類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

佇列、排程器與派遣器的分工

Ready queue 收集可執行但尚未分到 CPU 的工作；事件或裝置等待佇列則保存正在等待條件成立的工作。短期排程器從 ready 工作中選擇執行流，派遣器（dispatcher）負責交接並恢復上下文。長期准入與中期暫停分別管理工作進入系統及暫時移出記憶體；實際系統不一定各有獨立元件。

模式切換不一定切換行程；同一行程的系統呼叫就可能只有 user/kernel mode 的轉換。同一行程中的執行緒切換可更換暫存器與堆疊，卻共用使用者位址空間。

例如 P1 執行時發出磁碟請求並阻塞，P2 從 ready 變成 running。請求完成使 P1 回到 ready；若採非搶占式策略，P2 可以繼續執行。重複完成通知不得讓同一工作多次進入 ready queue，也不能讓同一執行流同時在兩顆 CPU 上執行。

若每輪有 10 ms 的有用工作，另花 1 ms 排程與派遣，總時間為 11 ms，額外負擔占 **1/11**。比值 1/10 則是相對於有用工作量的額外成本，兩者分母不同。此固定成本模型沒有計入快取與 TLB 干擾。

喚醒遺失：檢查之後仍有競賽

等待者先檢查 `done=false`、登記並釋鎖，即使之後看到自己仍是 WAITING，也不能直接保證安全：完成事件可能恰好發生在這次檢查之後、實際睡眠之前。

時序	等待者	完成方
1	釋鎖後讀到 WAITING	—
2	尚未睡眠	設 `done=true` 並發送喚醒
3	依舊執行先前決定，進入睡眠	不再有完成事件

如果排程器沒有與喚醒協調，第三步仍可能永久等待。要排除此交錯，必須由同步原語保證「釋鎖並進入等待」對喚醒者不可分割，或以等價的排程狀態協定保存提早到達的喚醒。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

以原子等待連接條件與喚醒

在同一把鎖 `L` 下保護條件與等待佇列。等待者取得 `L`，只要 `done=false`，就登記後呼叫 `park_and_release(L)`；返回時重新取得 `L`，再以 **while** 檢查條件。完成方取得 `L`，設定 `done=true`，喚醒等待者，再釋放 `L`。

`park_and_release` 是抽象原語：相對於持 `L` 的完成方，釋鎖與進入等待的交接必須不可分割；若喚醒發生於交接期間，不得遺失。它不是一般的 `unlock(); sleep()`。實作可使用具有此契約的 condition variable 或核心等待佇列／排程器協定。

此處的完成方可在允許取得 `L` 的執行環境中執行；若是真正的硬體中斷處理常式，不能使用可能睡眠的 mutex 或 condition variable，需採用適合中斷情境的鎖及喚醒機制，或將工作延後。

完成先取得鎖時，等待者看到 `done=true` 不睡；等待者先取得鎖時，完成方只能在登記與原子交接後設定條件並喚醒。被喚醒只代表具備執行資格；排程器仍要選取它。迴圈亦處理虛假喚醒及條件被其他執行緒改變。

CPU 密集與 I/O 密集的工作

假設工作 A 交替執行 CPU 1 ms 與等待 I/O 9 ms；工作 B 需要 CPU 10 ms。在單 CPU 且 I/O 可獨立進行的模型中，A 等待 I/O 時，CPU 可以執行 B。

CPU-bound 表示工作經常受限於運算資源；**I/O-bound** 表示它經常等待 I/O。這是工作負載的特性，不是行程永遠不變的身分。增加可執行工作可能改善 CPU 利用率，也可能使裝置佇列或記憶體過載。

若各行程的工作集超出可用頁框，頻繁缺頁可能導致 thrashing，增加行程數反而降低吞吐量。暫停與重新載入是中期資源管理；每次從 ready queue 選擇下一個工作則是短期排程。第 6 章會用共同工作負載比較具體排程策略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

fork、exec、exit、wait 各做什麼？

`fork` 建立子行程；子行程有自己的行程與位址空間管理資訊，也取得必要的資源引用。私有記憶體頁可透過寫入時複製（copy-on-write，COW）延後實體複製。

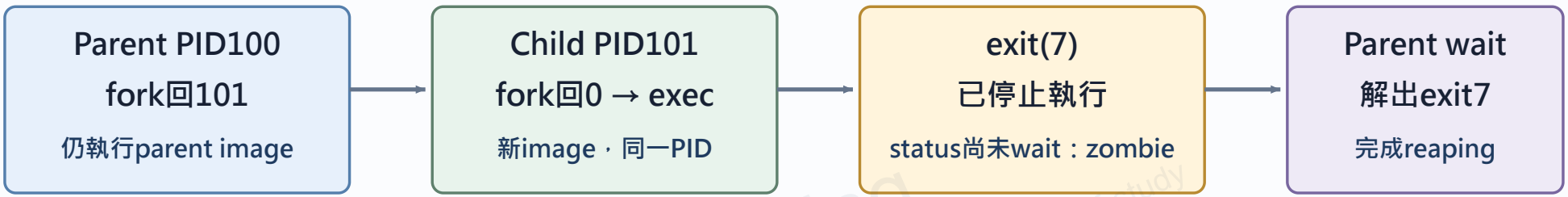
`exec` 以新程式映像取代呼叫者的映像，成功時不會回到原呼叫點，也不會新增行程。`exit` 結束執行並釋放大部分資源；父行程用 `wait` 取得子行程的結束狀態，完成回收（reaping）。

例如 PID 100 執行 `fork`：父行程得到子行程 PID 101，子行程得到 0。子行程成功執行 `exec` 後仍是 PID 101；若它以 7 結束，父行程應先確認正常結束，再從 `wait` 的狀態解出 7。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

fork、exec、exit、wait 各做什麼？ （續）

PID101：fork建立、exec換image、exit留status、wait收取



Parent先退出而child仍活著：orphan關係，不是上述已退出的zombie狀態。

箭頭：操作/生命週期順序。fork/exec均假設成功；不是four processes。

Zombie 是已結束但狀態尚未被回收的子行程；orphan 則是父行程先結束、仍存活的子行程。重新指定父行程與回收的細節取決於作業系統及其命名空間與 subreaper 政策。

兩次 fork 為何產生四個行程？

假設兩次 fork 都成功、沒有提前結束，且每個行程都會執行下一行。第一次由 P0 建立 P1；第二次由 P0 建立 P2，也由 P1 建立 P3。

已完成的呼叫	執行該呼叫的行程	新增子行程	存活總數
初始	P0	無	1
第一次 fork	P0	P1	2
第二次 fork	P0、P1	P2、P3	4

一般而言，若每個現有行程均執行下一次成功的 fork，則 N(0)=1、N(k+1)=2N(k)，由歸納法得 N(k)=2^k。若只有原父行程繼續、子行程提早 exec 或結束，或者 fork 失敗，就不能套用此式。例如只有 P0 再建立一個子行程時，總數為 3。這棵樹也不決定實際 PID 或輸出先後。

繼承描述符與獨立開啟檔案

假設父行程的 fd 3 引用開啟檔案紀錄 O1，其 offset 為 0。fork 後父子各有描述符表項，但兩個表項仍可引用同一個 O1，因此共享 offset。

在檔案足夠長、沒有其他操作且每次確實讀到指定數量的條件下，父行程讀 100 bytes，offset 由 0 變 100；子行程接著讀 50 bytes，從 offset 100 開始，最後變成 150。若另外 open 同一檔案，新的開啟紀錄 O2 可有獨立的 offset。這與使用者記憶體的 COW 是不同的共享關係。

fork 失敗應走錯誤處理，不能誤認為子行程。exec 失敗通常回傳錯誤，但特定實作跨過不可回復點後，不能無條件假設舊映像完整。父行程若從不等待已結束的子行程，可累積未回收的結束狀態。

多執行緒行程呼叫 fork 時，子行程只保留呼叫者這條執行緒；複製過去的鎖狀態可能沒有存活的持有者。子行程在 exec 前可安全執行的操作須遵守系統介面的限制。

共享記憶體與訊息傳遞

共享記憶體先建立共同的儲存物件與映射，再由各行程讀寫資料；仍需權限檢查與同步。訊息傳遞則透過 `send`／`receive` 交換資料，其等待、容量與複製行為由介面契約決定。

必須約定的事項	例子	不釐清的風險
對象與命名	行程、端點或 mailbox	權限、識別碼重用與清理
等待	send／receive 各自可否阻塞	send 完成可能只代表訊息已入列
容量	零容量、有界或理想無界	緩衝區滿時的 backpressure；實體資源仍有限
資料邊界	完整訊息或 byte stream	一次 read 不一定對應一筆紀錄
生命週期	close、unlink、remove、對方失敗	EOF、錯誤與佇列消失的語意不同

例如容量為 2 的抽象通道先接收 J1、J2，接著 J3 必須等到接收者取走 J1 才能入列。即使兩端都使用阻塞呼叫，只要 send 完成代表「成功入列」，它就不一定是零容量 rendezvous。

單生產者單消費者環形佇列

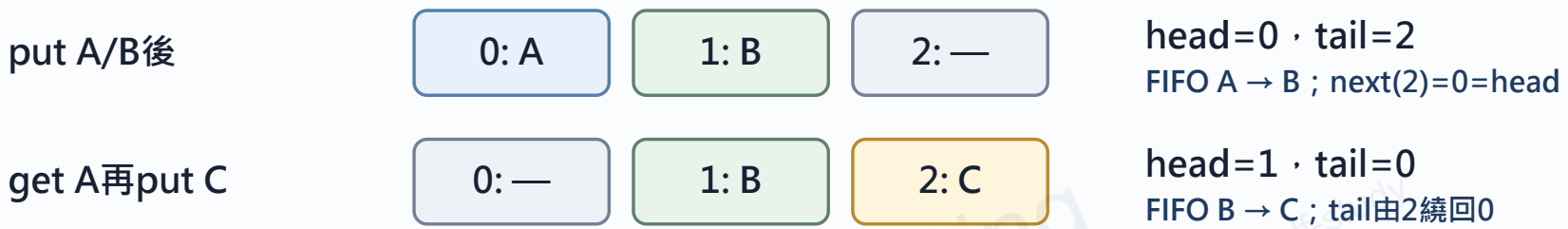
一個 producer 和一個 consumer 共用三個槽位 `slots[3]`，以及初值均為 0 的原子索引 `head`、`tail`。`head` 指向下一筆待讀資料，`tail` 指向下一個待寫位置。保留一格後，空佇列滿足 `head=tail`；滿佇列滿足 `next(tail)=head`，其中 `next(x)=(x+1) mod 3`。因此容量是 2，而非 3。

事件	head	tail	FIFO 中尚未取出的資料	結果
初始	0	0	[]	空
放入 A	0	1	[A]	發布 tail＝1
放入 B	0	2	[A,B]	滿，next(2)＝0＝head
嘗試放入 C	0	2	[A,B]	佇列滿，不能發布 C
取出 A	1	2	[B]	釋放槽位 0
放入 C	1	0	[B,C]	tail 繞回，仍為滿
依序取出 B、C	0	0	[]	空

單生產者單消費者環形佇列 (續)

SPSC ring：3 slots保留1格；full時禁止覆寫

head=下一個待取，tail=下一個待寫；empty h=t，full next(t)=h。



write item → release tail；read item → release head

灰：邏輯空槽（舊bytes可仍在）。彩色：尚未消費items。箭頭/文字表示publish或reuse順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發布資料與重用空間的先後關係

Producer 以 acquire 讀取 `head`；確認有空間後，先寫入 `slots[tail]`，再以 release 發布新的 `tail`。Consumer 以 acquire 讀取 `tail`；確認有資料後，先讀取 `slots[head]`，再以 release 發布新的 `head`。只有 producer 寫 `tail`，只有 consumer 寫 `head`。

第一條先後關係：slot 寫入先於發布 `tail`；consumer 的對應 acquire 觀察到該發布後，才能安全讀取已寫完的資料。第二條：slot 讀取先於發布 `head`；producer 的對應 acquire 觀察到釋放後，才能重用該槽位。空／滿檢查與唯一寫入者的前提共同維持容量 0 至 2 及 FIFO 次序。

普通共享整數不能替代原子操作與記憶體順序：C/C++ 中衝突的非同步存取可能構成 data race。若先發布 `tail` 再寫資料，讀者可能看見舊內容；若增加第二個 producer，也必須重新設計索引競爭。跨行程使用原子物件仍須平台明確支援。有限追蹤能檢查索引，不能代替同步論證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從位元組串流組出完整紀錄

假設每筆紀錄以 2-byte big-endian 長度開頭，後接 payload；payload 最多 8 bytes。兩筆資料 `ABC`、`DE` 編成：

```
00 03 41 42 43 | 00 02 44 45
```

串流中的直線僅供說明，不會由 `read` 保證。三次讀取可能分別得到 `[00]`、`[03 41 42]`、`[43 00 02 44 45]`。解析器先累積兩個 header bytes，解出長度 3，再累積 `ABC` 並輸出；接著解出長度 2，累積並輸出 `DE`。

如果 EOF 出現在半個 header，或 payload 尚未收齊時，就是截斷（truncation），不能當成完整訊息。收到長度後先驗證不超過 8，並檢查配置大小的算術，再接收 payload。TCP 與 pipe 的 byte stream 不替應用程式劃分紀錄；datagram 與 message API 則須依各自的邊界及截斷規則處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享記憶體的名稱、描述符與映射

建立者以名稱 `/course-ring` 建立物件、設定長度為 4096 bytes，再以 `MAP_SHARED` 映射；另一個行程以相同名稱開啟及映射。假設 P1 的映射起點是 0x100000，P2 是 0x700000，則物件 offset 16 分別對應虛擬位址 0x100010 與 0x700010。兩者仍可能存取同一個 backing 的位置。

因此共享結構可儲存 offset 或 index；不應將 P1 的絕對指標直接交給 P2。關閉 descriptor 不會自動解除既有映射；`shm_unlink` 移除名稱，阻止後續透過舊名稱開啟，但現有引用與映射可以繼續使用，直到各自生命週期結束。

`MAP_SHARED` 本身不保證初始化先於讀取。建立者應完成物件大小、版本、長度與欄位初始化，再以支援跨行程的同步機制發布 ready 條件。消費者不能假設對尚未設定大小、超出有效範圍的區域存取只是讀到零；固定 sleep 一秒也無法保證發布順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

移除 System V 與 POSIX 佇列

物件	移除操作	已有使用者受到的影響
POSIX message queue	`mq_unlink` 移除名稱	已開啟的引用仍可使用，直到最後引用關閉後才銷毀
System V message queue	`IPC_RMID` 移除佇列	阻塞中的 send／receive 被喚醒並收到 `EIDRM`
自訂應用通道	停止傳送、排空，最後銷毀	只能依應用程式明定的狀態機推理

不能把「移除前必定等待消費者排空」當作這兩種原生 API 的共同保證；移除名稱也不等於自動向每個接收者發送 EOF。若需要排空與正常停止，應用程式必須另行約定協定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用結束訊息通知消費者

假設一個 producer 和一個 consumer 透過容量為 4 的 FIFO 交換工作，正整數是合法工作，0 保留為 STOP。Producer 先送 1、2、3，停止接受新工作後再送 0：佇列依序為 `[1]` → `[1,2]` → `[1,2,3]` → `[1,2,3,0]`。

Consumer 依次完成 1、2、3，再收到 STOP 並回覆確認；管理者收到確認後才清理資源。這是應用層協定，並非核心自動提供的排空保證。

若 0 也是合法工作，應改用帶有明確類型標記的訊息。若有兩個 consumer 而只送一個 STOP，另一個可能永遠等待；可以各送一個 STOP，或設計關閉狀態與廣播。若 consumer 在確認前故障，還須另外規定恢復或重派，不能由正常路徑推論工作恰好完成一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mach：連接埠權限與訊息順序

以教材的歷史 Mach port 模型為例，server 持有 receive right，client 需要對應的 send right。訊息 header 指明目的地與回覆 port；帶型別的內容可包含資料或受控的權限。知道 port 的本地名稱不代表擁有接收權，也不能把它當成全系統通用的指標。

假設 client A 依序送出 A1、A2，client B 送出 B1。在本模型中，同一 sender 的 A1 應先於 A2，但不同 sender 之間沒有自動建立全域順序；例如 A1、B1、A2 或 B1、A1、A2 都可能出現。Server 回覆 A1 時，A2 與 B1 仍可能等待；send 成功也不等於 reply 已成功收到。

佇列滿時是等待、逾時還是立即失敗，須依所選操作的契約判定。Out-of-line memory 可透過虛擬記憶體映射或 COW 減少大資料複製，但不能因此宣稱所有訊息與跨主機傳輸都不複製。權限和共享記憶體的生命週期仍需由協定處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows ALPC 的連線與私有通道

依原書的歷史 Windows 案例，server 公開 connection port；取得授權的 client 送出連線請求，server 接受後建立雙方可用的通訊端點。Connection port 用於建立通道，並非每傳送一筆資料就重建一次連線。

教材以小訊息複製與大資料使用 shared section 區分路徑，並以當時案例的 256 bytes 為分界；這不是所有現代 Windows 版本固定的 API 上限。在該模型下，128-byte 請求可採小訊息路徑，4096-byte payload 可放入預先建立的 section，再以 offset 和 length 指示範圍。

即使用 section 減少重複複製，仍須驗證映射範圍、權限及可寫入者，不能信任 client 任意提供的指標。應用程式通常透過公開 RPC 等介面間接使用此類本機機制；Mach、ALPC 和 POSIX queue 的命名、關閉與權限語意不能直接互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Socket 端點與 RPC stub 的職責

Socket 端點由位址家族、位址與 port 等資訊識別；常見的串流伺服器先 `bind`、`listen`，再用 `accept` 取得連線端點。原本的 listening socket 仍可接受後續連線。Port 不等於 PID；連線成功也不表示應用程式的操作已完成。

RPC client stub 將操作與參數編碼為訊息；server stub 驗證並解碼訊息、呼叫服務，再編碼結果。不同機器的整數寬度、endianness、對齊方式及指標值不能直接當成共通的 wire format。

例如規定 request ID 為 32 bits、opcode 為 16 bits、amount 為 32 bits，全部使用 big-endian。ID＝77、opcode＝1、amount＝25 時，10 bytes 為 `00 00 00 4D | 00 01 | 00 00 00 19`。接收者先驗證長度與 opcode，再解碼數值；傳送 client 的指標位址不會讓 server 自動取得該記憶體的存取權。

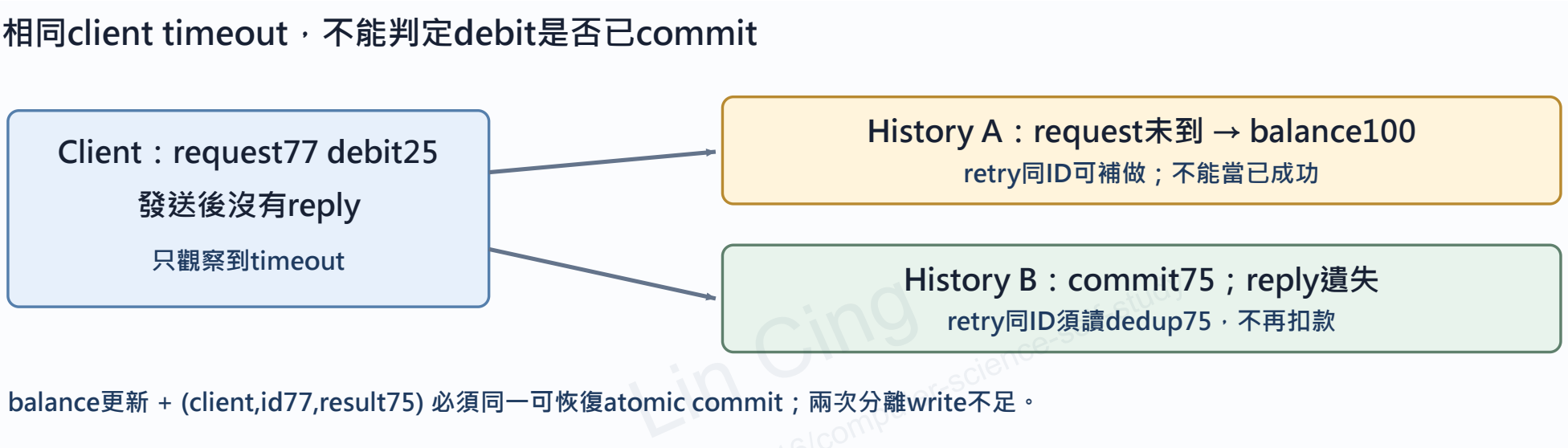
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逾時：兩種不同的執行歷程

帳戶原有 100，request 77 要扣款 25。情況 A：請求沒有到達 server，餘額仍為 100；情況 B：server 已提交扣款、餘額為 75，但回覆遺失。Client 在兩種情況下都只觀察到 timeout。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逾時：兩種不同的執行歷程 （續）



因此，改用新 ID 盲目重試可能讓情況 B 再扣一次、變成 50；完全不重試則可能讓情況 A 的操作遺漏。若要重試且避免重複扣款，server 可用 `(client identity, request ID)` 查詢去重紀錄，並把餘額更新與結果紀錄放在同一個可恢復的原子交易中。

第一次 request 77 將 100 改為 75 並記錄結果 75；重送同一 ID 時回傳 75，不再扣款。並行的重複請求也須由交易或唯一性約束協調，不能都先查到「不存在」再各自扣款。

至多執行一次與可用性

在去重紀錄有效的期間，同一 request ID 可以限制為至多一個效果。若先扣款、再另外寫入紀錄，兩者之間故障就會留下重複執行的機會；若紀錄只存在 RAM，重啟後也會消失。紀錄何時可以刪除，必須與重試期限、ID 作用範圍及 epoch 等規則配合。

At-most-once 允許請求永久遺失、一次也未執行。At-least-once 在持續重試與最終可達等條件下提高完成機會，但可能重複非冪等操作；扣款 25 就不是冪等操作。若要談 exactly-once，必須明確界定交易、持久紀錄、故障與通訊假設，不能只靠名稱保證。

Pipe 和 named FIFO 支援本機串流；socket 可支援更一般的端點及跨主機通訊。應用仍須處理 framing、backpressure 與對方失敗；IPC 成本也包括喚醒、同步、引用與重試。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

行程與資料的生命週期

假設 producer P 與 consumer C 使用容量 2 的 FIFO。P 送入 J1、J2 後，嘗試送 J3 時因佇列已滿而等待；C 取走 J1，釋出一個槽位並喚醒 P；P 回到 ready，在被選中執行後才送入 J3。過程中 FIFO 次序不變，容量也不超過 2。

取出訊息不表示工作的效果已經持久完成；行程結束也不代表父行程已經以 `wait` 收取狀態。PID 與 PCB 描述執行，描述符表項引用開啟檔案紀錄，共享記憶體 offset 指向資料，而 request ID 用來辨認重試。這些身分與生命週期必須分開追蹤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

檢查安全性的關鍵條件

行程切換時，保存與恢復必須對應同一條執行流；等待與喚醒必須圍繞同一個受同步保護的條件。環形佇列須先寫入資料再發布、讀取完才允許重用空間；串流解析器只交出完整且長度合法的紀錄；RPC 的實際效果與去重結果必須一同提交。

逐步範例用來核對中間狀態；不變量則解釋更多允許的事件順序為何仍然安全。有限測試不能證明所有執行交錯或故障都正確。若更換 API、增加生產者或消費者、改變記憶體順序或持久化策略，就要重新檢查各項前提。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 3. read(2); pipe(7); msgctl(2); mq_unlink(3).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study